

1. Regex

1. C'est quoi ce machin ?

```
^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[-+!*@#$$%^&]).{8,}$
```

Une Regex (abréviation d'Expression Régulière, ou Regular Expression en anglais) est une chaîne de caractères spéciale qui sert de modèle (pattern) pour rechercher, capturer ou modifier du texte.

Voici à quoi ça sert concrètement :

- Valider une saisie : Vérifier si ce qu'a tapé un utilisateur ressemble bien à un e-mail, un numéro de téléphone ou un code postal valide.
- Rechercher du texte complexe : Trouver tous les mots qui commencent par une majuscule, font 5 lettres et se terminent par un "s".
- Extraire des informations : Récupérer automatiquement tous les prix ou toutes les dates présents dans un gros volume de texte (comme une facture ou un article de journal).
- Nettoyer des données : Supprimer tous les espaces en trop ou masquer les numéros de carte bancaire dans un fichier de logs.

2. Avec Python ?

Python propose le module `re` permettant d'intégrer facilement les regex dans son code.

```
import re
```

Préfixer une chaîne avec `r` permet au système d'interpréter une chaîne comme étant un pattern `regex`.

- évite doubler les `\`
- selon les `IDE`, améliore la coloration syntaxique

```
pattern = r'^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[-+!*@#$$%^&]).{8,}$'
```

match

Vérifie si le motif correspond au tout début de la chaîne.

```
m1 = re.match(r'hello', 'hello world !')
print(m1) # <re.Match object; span=(0, 5), match='hello'>
```

```
m2 = re.match(r'world', 'hello world !')
print(m2) # None
```

search

Parcourt toute la chaîne et s'arrête dès qu'une correspondance est trouvée, peu importe où elle se trouve.

Idéal pour savoir si un motif existe quelque part dans un texte.

Ne renvoie que la première occurrence ou None si rien n'est trouvé.

```
m1 = re.search(r'world', 'hello world !')
print(m1) # <re.Match object; span=(6, 11), match='world'>

m2 = re.search(r'world', 'hello khun !')
print(m2) # None

m3 = re.search(r'hello', 'hello world, hello !')
print(m3) # <re.Match object; span=(0, 5), match='hello'>
```

findall

Parcourt toute la chaîne et récupère toutes les occurrences non chevauchantes.

- Si votre regex n'a pas de parenthèses () : renvoie une liste de chaînes (le texte complet capturé)
- Si votre regex contient un groupe () : renvoie une liste de chaînes contenant uniquement le contenu du groupe.
- Si votre regex contient des groupes (...) : renvoie une liste de tuples contenant uniquement le contenu des groupes.

```
m1 = re.findall(r'<p>.+?</p>', '<p>Hello</p><p>world!</p>')
print(m1) # ['<p>Hello</p>', '<p>world!</p>']

m2 = re.findall(r'<p>(.*?)</p>', '<p>Hello</p><p>world!</p>')
print(m2) # ['Hello', 'world!']

m3 = re.findall(r'<(.*?)>(.*?)</.*?>', '<p>Hello</p><span>world!</span>')
print(m3) # [('p', 'Hello'), ('span', 'world!')]
```

Remarque: les symboles `.`, `+` et `?` seront expliqués plus en détails dans les chapitres suivants.

3. Les "Wildcards" et les Classes de Caractères

Remplace n'importe quel caractère

```
m1 = re.findall(r'..mm.', 'l\'homme et la femme')
print(m1) # ['homme', 'femme']
```

Les classes personnalisées

Accepte tous les caractères entre [et]

```
m1 = re.findall(r'l[aeu]\b', 'le monsieur a lu le journal sur la plage')
print(m1) # ['le', 'lu', 'le', 'la']
```

Les intervalles

Accepte tous les caractères entre 2 caractères

```
m1 = re.findall(r'[a-e][0-5]', 'a2 b3 b0 b6 f5')
print(m1) # ['a2', 'b3', 'b0']
```

Les classes prédéfinies

- \d : Un chiffre (équivalent à [0-9])
- \w : Un caractère alphanumérique + underscore (équivalent à [a-zA-Z0-9_])
- \s : Un espace blanc (espace, tabulation, nouvelle ligne)

Leurs versions majuscules (\D, \W, \S) sont leurs exacts contraires.

4. Les Quantificateurs (Combien de fois ?)

- * : 0 fois ou plus.
- + : 1 fois ou plus.
- ? : 0 ou 1 fois (rend un caractère optionnel).
- {n}, {n,}, {n,m} : Répétition précise.

```
m1 = re.findall(r'..m{2}.', 'l\'homme et la femme')
print(m1) # ['homme', 'femme']

m2 = re.findall(
    r'<.+>.+</.+>',
    ...
    <p>paragraphe</p>
    <span></span>
    <h1>titre</h1>
    ...
)
```

```

)
print(m2) # ['<p>paragraphe</p>', '<h1>titre</h1>']

m3 = re.findall(
    r'<.+.?.*</.+>',
    ...
    <p>paragraphe</p>
    <span></span>
    <h1>titre</h1>
    ...
)
print(m3) # ['<p>paragraphe</p>', '<span></span>', '<h1>titre</h1>']

```

Le piège du "Greedy vs Lazy" Pourquoi `.*` ou `.+` peut parfois capturer tout votre texte et comment le calmer avec `.*?` ou `.+?`.

```

m1 = re.findall(
    r'<.+.?.*</.+>',
    '<p>paragraphe</p><span></span><h1>titre</h1>'
)
print(m1) # ['<p>paragraphe</p><span></span><h1>titre</h1>']

m2 = re.findall(
    r'<.+?>.??</.+?>',
    '<p>paragraphe</p><span></span><h1>titre</h1>'
)
print(m2) # ['<p>paragraphe</p>', '<span></span>', '<h1>titre</h1>']

```

5 : Les Ancres et les Frontières

- `^` : Début de ligne / chaîne.
 - `$` : Fin de ligne / chaîne.
 - `\b` : Frontière de mot (indispensable pour chercher "tombe" sans trouver "tombeau").
-

6. Les Assertions conditionnelles

- look ahead (`?=...`) tous ce qui est suivi par
 - negative look ahead (`?!...`) tous ce qui n'est suivi pas par
 - look behind (`?<=...`) tous ce qui est précédé par
 - negative look behind (`?<!...`) tous ce qui n'est précédé pas par
-

7. Les Flags (Options de configuration)

Les flags permettent de modifier le comportement par défaut du moteur de regex. En Python, on les passe en deuxième ou troisième argument des fonctions `re` (ex: `re.search(pattern, texte, re.IGNORECASE)`).

```
# Sans le flag
print(re.findall(r'python', 'Python est cool')) # []

# Avec le flag
print(re.findall(r'python', 'Python est cool', re.IGNORECASE)) # ['Python']
```
